Adding More Bugs to a Task

This guide explains how to inject additional bugs into a task so the model has more to solve. There are two parts:

- **Part A — Author the bug** (clone the repo in the sandbox, edit code, export the diff).
- **Part B — Wire it into the task** (drop the diff into `assets/bugs/`, update the `Dockerfile`, add a detecting test + fix).

The repo URL, commit hash, and file paths in the commands below are examples taken from this task (`apache/superset`). Swap in the values from your own task's `task.toml` (`repo_url`, `base_commit`) and the files you're editing.

Keep in mind that this guide uses the superset repository as an example. Make sure you modify the commands according to your task (e.g., repo URL, commit hashes).

How it works (mental model)

The model receives the repo exactly as it exists after the Docker build, snapshotted as the `harbor_base` git tag. Any source change baked in **before** that snapshot becomes a bug the model must find and fix.

- A **bug patch** turns good code into buggy code (`good -> buggy`). This is what the `Dockerfile` applies.
- A **fix patch** is the reverse (`buggy -> good`). This goes into `solution/fix.patch` so the oracle solution still scores 100%.
- A bug is only gradable if a **test** in `tests/test.patch` fails on the buggy state and passes once fixed (registered under `f2p` in `task.toml`).

Part A — Author the bug

Do this in stages — don't run everything at once. First clone and check out the right commit, **then** modify the code, and only after that export the diffs.

Run these from the task root (the directory that contains `Tasks/`). The repo is cloned into a `superset/` folder right there, so `Tasks/...` paths stay relative.

The repo URL, commit hash, and file paths below are **examples specific to this task** (`apache/superset`). Swap in your own task's `repo_url` and `base_commit` from `task.toml`, and the files you're editing.

1. Open a terminal in the sandbox and clone the repo, then check out the exact `base_commit` the `Dockerfile` uses:

```
Shell
git clone https://github.com/apache/superset

git -C superset checkout ff1cf5f24ed29835843853dcd5af9a27918fe6b5   # must match base_commit in task.toml
```

2. Now modify the codebase to introduce the bug.

Open the file(s) under `superset/` in your editor and make a real, deliberate defect — something a test can detect (wrong output, off-by-one, a dropped branch, etc.). For example, in this task you might edit:

```
None
superset/superset-frontend/plugins/plugin-chart-echarts/src/utils/formatters.ts
```

Take your time here: this hand-written change *is* the bug. Don't rush it into the same step as the commands below.

3. Once the bug is in place, export the two diffs (writing the bug patch straight into the task):

```
Shell
# Bug patch (good -> buggy): the Dockerfile applies this
git -C superset diff > Tasks/environment/assets/bugs/bug_01.patch


# Fix patch (buggy -> good): the reverse, kept for the oracle fix in step 4
```

```
git -C superset diff -R > fix_add.patch
```

The oracle needs a fix for your bug too. Since you broke known-good code, the fix is just reverting to that correct code — which is exactly what `git diff -R` captures above, so you normally don't hand-write a separate fix. You **must** still merge `fix_add.patch` into `Tasks/solution/fix.patch` (Part B, step 4) so the oracle repairs both the original and the new bug and still scores 100%. Exception: if the *correct* behavior differs from what the base commit had (not a pure regression), author the real fix by hand instead of using the reverse diff.

Don't forget the tests. A bug is only gradable if a test catches it. After authoring the bug, you **must** modify `Tasks/tests/test.patch` to add tests that fail on the buggy state and pass once fixed, and register their names under `f2p` in `task.toml` (see Part B, step 3). Skipping this means the model gets no signal or reward for solving the new bug.

Notes:

- **Pin the commit exactly.** Editing at `HEAD` or another commit may make the `git apply` in the `Dockerfile` fail or land in the wrong place.
- **Use the working-tree diff (uncommitted).** `git diff` compares your edits against the checked-out commit, which is exactly `good -> buggy`. If you commit first, diff against the commit: `git diff <base_commit>`.

Part B — Wire the bug into the task

1. Place the bug patch

Keep bug patches in `Tasks/environment/assets/bugs/` (only `.patch` files here, never test files):

```
None
Tasks/environment/assets/bugs/bug_01.patch
```

2. Update the `Dockerfile` to copy → apply → delete

The bug must be applied **before** the `harbor_base` snapshot, and the patch file must be **deleted** from the agent-visible dir so the model can't read the answer.

The clone URL and commit hash below are **examples specific to this task**. Adapt them (and keep the rest of the structure) to match your task's existing `Dockerfile`.

Shell

```
# [IMPORTANT] Copy bug patches to a TEMP path (not into /app) so they can be

# applied into the base snapshot and never reach the agent-visible tree.

COPY assets/bugs/ /tmp/bugs/


RUN git clone https://github.com/apache/superset /app \

 && git -C /app checkout ff1cf5f24ed29835843853dcd5af9a27918fe6b5 \

 && rm -rf /app/.git \

 # [IMPORTANT] Apply every bug patch BEFORE the snapshot below, so the defect

 # becomes part of harbor_base (the exact state the model starts from).

 && for p in /tmp/bugs/*.patch; do git -C /app apply "$p" || patch -p1 -d /app <
"$p"; done \

 && git -C /app init -q \

 && git -C /app -c user.email=h@l -c user.name=h add -A \

 && git -C /app -c user.email=h@l -c user.name=h commit -q -m base \

 # [IMPORTANT] harbor_base = base commit + all bug patches. Must come AFTER the

 # apply loop; the verifier resets to this tag.

 && git -C /app tag harbor_base


# ... npm install step unchanged ...


RUN mkdir -p /logs /app/.harbor_assets

COPY assets/ /app/.harbor_assets/

# [IMPORTANT] Delete the bug diffs from the agent-visible dir so the model
```

```
# can't read the answer (the reverse of a bug patch is the fix).

RUN rm -rf /app/.harbor_assets/bugs
```

Note: `rm` is enough for anti-cheat. The agent only sees the final merged filesystem, not intermediate build layers, so the deleted patch is unreadable even though it existed in an earlier layer.

3. Add a detecting test

`Tasks/tests/test.patch` is itself a diff that **adds** test cases to the repo's test file. To add a test for your new bug you grow that patch — and just like `fix.patch`, you don't hand-edit it, you regenerate it from the clone. The trick is to diff **only the test file** so your source-bug edits don't end up in it.

Run from the task root, reusing the `superset/` clone from Part A. The test-file path and the commit hash below are **examples specific to this task** — swap in your own task's `base_commit` (from `task.toml`) and the test file you're extending:

```
Shell
cd superset


# 1) Start from the tests the task already adds, so you extend rather than
#    overwrite them.
git apply ../Tasks/tests/test.patch


# 2) Open the test file and add your new test case(s), e.g.:
#
superset-frontend/plugins/plugin-chart-echarts/test/utils/formatters.test.ts
#    Each new test must FAIL on the buggy state and PASS once fixed.


# 3) Regenerate test.patch as the diff of ONLY the test file vs base_commit.
#    Diffing the explicit path keeps your source bug out of test.patch.
```

```
git diff ff1cf5f24ed29835843853dcd5af9a27918fe6b5 -- \

  superset-frontend/plugins/plugin-chart-echarts/test/utils/formatters.test.ts \

  > ../Tasks/tests/test.patch

cd ..
```

Then register the new test's name under `f2p` in `Tasks/task.toml` (tests that must keep passing go under `p2p`). The `f2p`/`p2p` names must match the test titles exactly, since the verifier keys the reward off them.

Never put source changes in `test.patch`. The verifier resets the `test.patch` paths to `harbor_base` and re-applies `test.patch`, so it must contain test files only — diffing the explicit test path (step 3 above) guarantees that.

4. Extend the oracle fix

This is where `fix_add.patch` (from Part A) gets folded into the oracle fix. `Tasks/solution/fix.patch` already fixes the original bug, and the oracle must now fix **both** the original and the new bug. This matters because `solve.sh` runs **only** `git apply /solution/fix.patch` — it never applies your bug patches or `fix_add.patch` — so your new fix won't take effect unless it lives inside `fix.patch`.

You do not hand-edit `fix.patch`. Don't open it and paste in hunks. Instead you let `git` regenerate the file: rebuild the exact buggy state the oracle runs against (`harbor_base` = base commit + all bug patches), apply every fix on top, then dump a fresh diff that **overwrites** `fix.patch`. Run from the task root, reusing the `superset/` clone from Part A. (The paths and file names below are **examples specific to this task** — substitute your own.)

```
Shell
cd superset


# 1) Recreate harbor_base = base + every injected bug, committed as baseline.

#    First discard your manual edits from Part A (already saved as patches).

git checkout -- . && git clean -fdq

git apply ../Tasks/environment/assets/bugs/*.patch
```

```
git commit -aqm buggy-baseline


# 2) Apply the fixes ON TOP of that buggy commit:
git apply ../Tasks/solution/fix.patch    # the existing oracle fix
git apply ../fix_add.patch              # your new fix (reverse of your bug)


# 3) Diff the fixed tree against the buggy commit -> ONE combined patch.
#    This line REPLACES fix.patch with the merged result. Git does the merge.
git diff > ../Tasks/solution/fix.patch
cd ..
```

After step 3, `Tasks/solution/fix.patch` **is** the merged patch: the diff between the buggy baseline and the fully-fixed tree. This works whether your bug is in the same file as the original fix or a different one.

Why regenerate instead of hand-merge? `fix.patch` must be a diff from the buggy `harbor_base` state, **not** the clean base commit — otherwise the hunks for the injected bug won't line up when `solve.sh` runs. Letting `git diff` produce it guarantees correct line numbers.

Shortcut (disjoint files only): if your new bug is in a completely different file than the existing fix, a `.patch` is just stacked `diff --git` blocks, so you can append instead of regenerating: `cat fix_add.patch >> Tasks/solution/fix.patch`. If the two fixes touch the same file, the stacked line numbers won't apply — use the regenerate method above.

5. Verify locally

```Shell
docker build -t task-env Tasks/environment


# Buggy state -> the new f2p test should FAIL
docker run --rm task-env bash -lc 'bash /tests/test.sh'
```

```
# After the oracle fix -> all tests should PASS

docker run --rm task-env bash -lc 'bash /solution/solve.sh && bash /tests/test.sh'
```

Key rules

- Apply bugs **before** the `git init … tag harbor_base` step, or the snapshot won't contain them.
- **Never** put test files in a bug patch. The verifier resets `test.patch` paths to `harbor_base` and re-applies its own tests, so those edits are discarded.
- Every bug needs a matching `f2p` test, or the model gets no reward for solving it.
- Keep `solution/fix.patch` in sync so the oracle still passes.
- Delete `assets/bugs/` from `/app/.harbor_assets/` in the `Dockerfile` so the diff never leaks to the model.

CONFIDENTIAL | qa_coder_92i_0609_62@outlier.ai+outlier | 2026-08-11T19:49:41.211Z